English Etymology Tagger: A Multi-Task Learning Approach to Etymology

Marcus Bennevall (& his trusty AI agents)
Stockholm University
marcusbennevall@gmail.com

Abstract

Predicting the etymological origin of words is a complex task involving both the identification of the source language and the entry mechanism, such as borrowing or inheritance. We present the English Etymology Tagger, a system that utilizes a custom dataset derived from Wiktionary and a Y-shape multi-task learning architecture. Our model integrates semantic embeddings with hashed orthographic features and is optimized using binary focal loss to address extreme class imbalance. We achieve an F1 score of 0.565 for source language prediction and 0.705 for entry mechanism classification on a held-out test set.

1 Introduction

Etymological analysis is fundamental to historical linguistics and lexicography, providing insights into how languages evolve and interact over centuries. However, automated tagging of etymological paths is challenging due to the overlapping nature of linguistic influences and the "heavy tail" distribution of source languages, where a few languages account for the majority of words while hundreds of others appear rarely.

We propose a neural architecture that jointly models the source language and the entry mechanism through multi-task learning. The full code and a demo are available on GitHub and HuggingFace (Hazelime, 2026; Bennevall, 2026a).

2 Dataset and Preprocessing

Our custom dataset is derived from the machine-readable Wiktionary dumps provided by Kaikki.org (Ylonen, 2022). We focus specifically on the 1,465,676 English word types, extracting the 102,111 entries containing etymological templates and remapping them into a structured format suitable for supervised learning. After filtering out irrelevant tags such as "Translingual," the final, publically available dataset contains 83,204 data points (Bennevall, 2026b).

2.1 Linguistic Consolidation

To reduce data sparsity and improve model convergence, we consolidate numerous language variants into parent families. For example, this involves merging specific historical stages, such as Late, Medieval, and Vulgar Latin, into a single "Latin" label. We also apply a frequency threshold of 1%, where any language accounting for less than this percentage of the total dataset is remapped to a collective "Other" category. The final label set consists of 23 source languages and 4 entry mechanisms: *borrowed*, *derived*, *calqued*, and *inherited*.

2.2 Imbalance Mitigation

The dataset exhibits severe class imbalance due to the dominance of Latin, French, and older variants of English in English etymology, each appearing in roughly a fifth of training data entries. To ensure the model does not ignore minority classes, we employ a two-pronged strategy. First, we randomly undersample 50% of the three high-frequency classes during training. Second, we apply inverse frequency weighting to the loss function. This scales the loss by a weight w proportional to N/N_{pos} (where N is the total samples and N_{pos} is the count of positive instances for a specific class), capped at a factor of 10.0 to prevent gradient instability.

3 Methodology

The core philosophy of our methodology is that a word's history is encoded in both its meaning and its form. We utilize a pipeline that converts raw strings into high-dimensional vectors, which are then processed through a shared neural trunk before splitting into specialized prediction tasks.

3.1 Feature Extraction

We utilize a hybrid feature representation with a total dimensionality of $d = 1324$. This vector is composed of two distinct parts. First, we use semantic embeddings consisting of 300-dimensional frozen fastText vectors (Bojanowski et al., 2017, wiki-news-300d-1M) – because fastText utilizes subword information, it is particularly effective at capturing the meaning of rare words. Second, we transform character n-grams (sequences of 3 to 5 characters), prefixes, and suffixes into 1024-dimensional features through orthographic hashing. This is essential for identifying morphological cues, such as the Latinate *-ation* suffix.

3.2 Model Architecture

We implement a Y-shape multi-task learning architecture with Multi-Layer Perceptron (MLP) classification using PyTorch (Caruana, 1997; Rumelhart et al., 1986; Paszke et al., 2019).

The shared trunk consists of a deep neural network with two hidden layers, Rectified Linear Unit (ReLU) activation functions, and a dropout rate of 0.4 to prevent overfitting. It reduces the input from 512 to 256 neurons and extracts features that are useful for both downstream tasks simultaneously.

Following the trunk, the model splits into two independent task-specific heads: one predicting source language with a hidden layer of 128 neurons and one predicting entry mechanism with a hidden layer of 32 neurons. Both conclude with a sigmoid output layer to facilitate multi-label classification.

3.3 Training and Optimization

In adherence with multi-task learning, we optimize the model using *combined* (binary) focal loss. Focal loss is a modification of standard cross-entropy that addresses class imbalance by down-weighting the loss contribution from "easy" (well-classified) examples in favor of "hard" examples (Lin et al., 2017). The total loss L_{total} is defined as:

$$L_{total} = \alpha \times L_{language} + \beta \times L_{mechanism}$$

To account for the higher complexity of the language classification, we weight its loss higher ($\alpha = 1.0, \beta = 0.5$). Each focal loss is defined as:

$$L(p_t) = -(1 - p_t)^{\gamma} \log(p_t)$$

Here, p_t represents the model's estimated probability for the ground truth class. An increase to γ leads to a reduction of the relative loss for well-classified examples ($p_t > 0.5$), forcing the optimizer to prioritize the minority languages that the model currently finds difficult to distinguish. We set γ to 2.0, as recommended by Lin et al. (2017).

4 Evaluation

We evaluated the model on a held-out test set of 20,422 samples. The results in Table 1 indicate that the shared trunk successfully captures underlying etymological signals that benefit both tasks. The higher performance on the entry mechanism task (F1: 0.705) compared to the language task (F1: 0.565) likely reflects a gap in task complexity due to differing number of classes (23 vs. 4).

Task	Precision	Recall	F1-Score
Language	0.586	0.545	0.565
Mechanism	0.684	0.727	0.705

Table 1: Model Performance Metrics

5 Conclusion

We have demonstrated that a Y-shape multi-task learning architecture effectively models the multi-faceted nature of word etymology. By combining semantic embeddings with orthographic hashing and utilizing binary focal loss, the system provides a robust baseline for automated etymological tagging. This approach offers a balanced solution to the significant challenge of linguistic class imbalance, allowing the identification of both dominant and minority etymological influences.

6 Reflection on AI Collaboration

I, Marcus, think this project has exposed me to both the highs and lows of AI coding collaboration. On the downside, it has made me keenly aware of weekly token quotas, overloaded servers, and models that introduce more bugs than they resolve. On the upside, however, it has given me access to a whole new world of coding. Now I do not only understand more vibe-coding memes, but also feel more confident in building things that are actually useful rather than simply being proofs of concept.

More concretely, one useful lesson that I have learned is to separate planning from building, since using high-intelligence models for building is often a waste of tokens, while using low-intelligence models for planning can lead to high-level decision errors with critical downstream ramifications.

References

Marcus Bennevall. 2026a. English Etymology Tagger. https://huggingface.co/spaces/MarcusBennevall/EtymologyTagger. Accessed: 2026-05-09.

Marcus Bennevall. 2026b. Etymology Tagger Dataset. https://huggingface.co/datasets/MarcusBennevall/EtymologyTaggerDataset. Accessed: 2026-05-09.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.

Rich Caruana. 1997. Multitask learning. *Machine learning*, 28(1):41–75.

Hazelime. 2026. Etymology-Tagger. https://github.com/Hazelime/Etymology-Tagger. Accessed: 2026-05-09.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(2):298–307.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, and 1 others. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. 1986. Learning representations by back-propagating errors. *Nature*, 323:533–536.

Tatu Ylonen. 2022. Wiktextract: Wiktionary as machine-readable structured data. In *Proceedings of the 13th Language Resources and Evaluation Conference*, pages 1317–1325.